

CLASSICAL BITS AND INFORMATION FOUNDATIONS

Binary representation, Boolean logic, classical circuits, reversible computation, communication, entropy, and the bridge to qubits

A detailed reference for quantum computing, quantum cryptography, and post-quantum foundations.

Topic 8 in the Quantum Mathematics Foundation series

How to use this volume

This chapter is designed as a bridge. It consolidates the classical information concepts that quantum computing assumes: binary state descriptions, Boolean logic, circuits, reversible maps, error models, and elementary information theory. The goal is not to repeat introductory programming material, but to make the structures explicit enough that quantum gates, measurement outcomes, QKD post-processing, and post-quantum algorithms feel natural.

Learning objectives

- Represent integers, states, and registers as bit strings.
- Use Boolean operations, truth tables, and binary arithmetic.
- Understand Hamming distance, parity, and elementary error correction.
- Distinguish deterministic, stochastic, reversible, and irreversible transformations.
- Analyze CNOT and Toffoli as classical reversible gates before treating them as quantum gates.
- Use entropy and channel models to describe uncertainty and noise.
- Explain precisely how a classical bit differs from a qubit.

Contents

1	1. Why classical bits still matter
2	2. Bits, binary states, and physical encodings
3	3. Bit strings and notation
4	4. Binary numbers and conversions
5	5. Signed integers and complements
6	6. Binary arithmetic
7	7. Boolean algebra
8	8. Logic gates and truth tables
9	9. XOR and arithmetic over $\text{GF}(2)$
10	10. Combinational circuits and adders
11	11. Bitwise operations, masks, and shifts
12	12. Hamming weight and Hamming distance
13	13. Deterministic state transformations
14	14. Reversible and irreversible computation
15	15. Reversible gates: NOT, CNOT, Toffoli, SWAP
16	16. Fan-out and copying
17	17. Random bits and distributions
18	18. Information and binary entropy
19	19. Classical channels and noise
20	20. Parity and repetition codes
21	21. Correlation and mutual information
22	22. Classical cryptographic primitives
23	23. Classical control in quantum systems
24	24. Classical bits versus qubits
25	25. Worked examples
26	26. Common mistakes
27	27. Exercises
28	28. Selected solutions
29	29. Reference sheet

1. Why classical bits still matter

Quantum computing does not replace classical information. Quantum processors are prepared, controlled, calibrated, measured, and interpreted using classical systems. A quantum algorithm normally begins with classical input, performs a quantum transformation, and ends with classical measurement outcomes.

A bit is simple; information systems emerge from layers

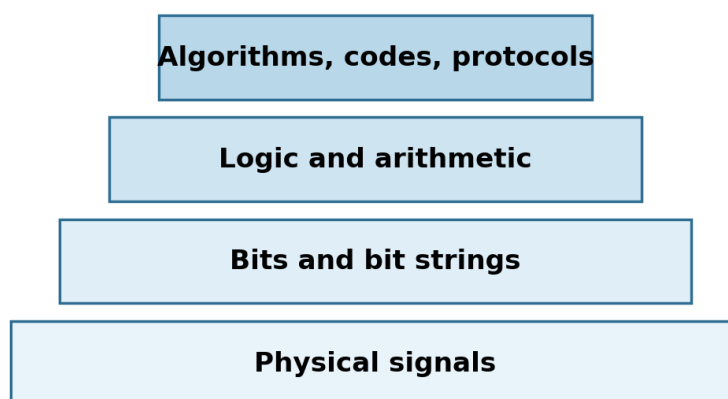


Figure 1. Information systems are built in layers, beginning with physical states and bits.

Classical foundations appear throughout quantum work:

- A measurement produces classical bits.
- Circuit parameters, feed-forward conditions, and error-decoder outputs are classical data.
- QKD uses classical authentication, sifting, error correction, and privacy amplification around a quantum transmission stage.
- Post-quantum cryptography is implemented entirely with classical bits, words, polynomials, matrices, and modular arithmetic.
- Quantum error correction combines quantum states with large amounts of classical syndrome processing.

A clear distinction between classical and quantum information prevents common conceptual errors, especially the idea that a qubit is merely an unknown classical bit.

2. Bits, binary states, and physical encodings

A bit is an abstract system with two distinguishable logical states, labelled 0 and 1. The labels do not specify a particular physical implementation. A bit might be stored using low/high voltage, two magnetic orientations, light/no light, charge/no charge, or two stable transistor configurations.

bit state x belongs to $\{0,1\}$

The abstraction is powerful because logical operations can be designed independently of the underlying hardware. A reliable physical implementation needs two states that are sufficiently distinguishable and stable against noise.

Logical state versus probability distribution

A definite classical bit is either 0 or 1. If we do not know which one, we may describe our uncertainty using probabilities:

$$P(X=0)=p_0, P(X=1)=p_1, p_0+p_1=1.$$

This probability distribution represents incomplete knowledge or randomness. It is not a coherent superposition; it has no relative complex phase and cannot produce quantum interference.

Registers

A register is an ordered collection of bits. An n -bit register has 2^n possible definite states. For example, a three-bit register can be 000,001,...,111.

$$\text{number of } n\text{-bit strings} = 2^n$$

3. Bit strings and notation

A bit string is an ordered sequence

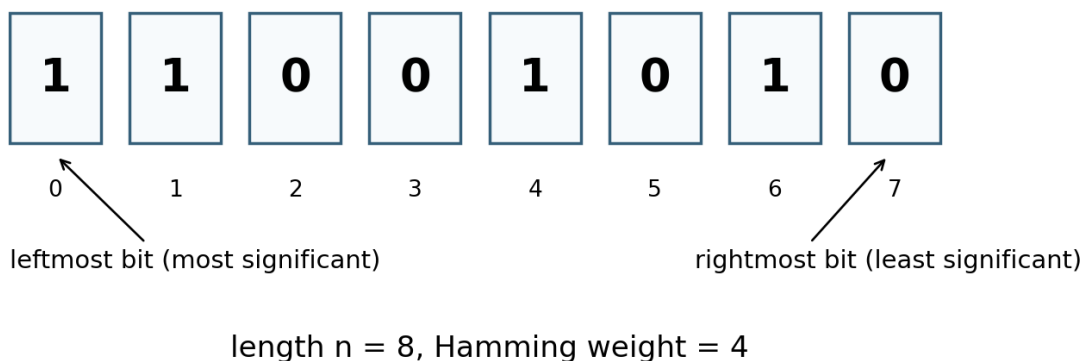


Figure 2. Bit strings are ordered; position and indexing conventions matter.

A bit string of length n is written

$$x = x_{n-1} x_{n-2} \dots x_1 x_0, \text{ each } x_j \text{ in } \{0,1\}.$$

The rightmost bit is commonly called the least significant bit (LSB), and the leftmost bit is the most significant bit (MSB). Programming languages and hardware descriptions may number bits from either end, so indexing conventions should always be stated explicitly.

Important quantities

- Length: the number of bit positions.
- Hamming weight $\text{wt}(x)$: the number of ones in x .
- Complement: flip every bit.
- Prefix and suffix: leading or trailing substrings.
- Concatenation xy : append y after x .

Example

$x=11001010$ has length 8 and $\text{wt}(x)=4$.

In quantum computing, the computational-basis state corresponding to this string is written $|11001010\rangle$. The ordering of qubits in software may differ from the order used when printing measurement strings; endianness must therefore be handled carefully.

4. Binary numbers and conversions

Binary place values

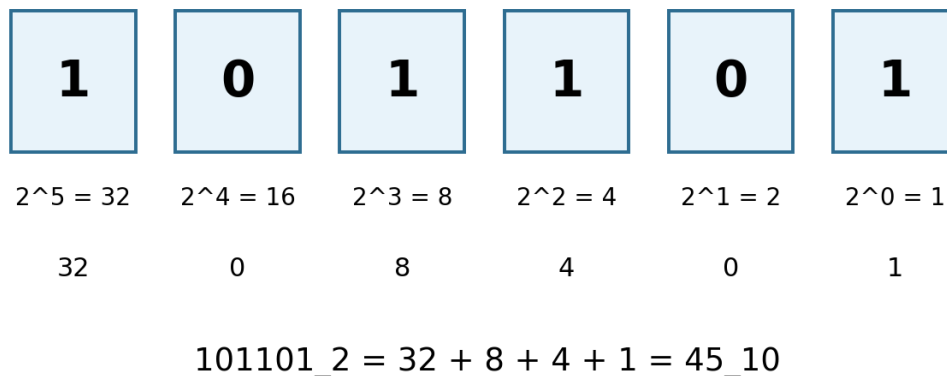


Figure 3. Each binary digit multiplies a power of two.

An unsigned binary string represents an integer using positional weights:

$$(b_{\{n-1\}} \dots b_{1b_0})_2 = \sum_{j=0}^{n-1} b_j 2^j.$$

Example

$$101101_2 = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2 + 1 = 45.$$

Decimal to binary

Repeatedly divide by two and record the remainders, or select powers of two greedily. For 45, choose 32, then 8, then 4, then 1, yielding 101101.

Hexadecimal

Hexadecimal compresses groups of four bits. The symbols 0-9,A-F represent values 0-15.

Binary	Hex	Decimal
0000	0	0
1001	9	9
1010	A	10
1111	F	15

Thus $1101\ 0110_2 = D6_{16}$. Hexadecimal is common in cryptography, machine code, memory addresses, and bit masks.

5. Signed integers and complements

Unsigned n -bit numbers represent 0 through $2^n - 1$. Signed arithmetic requires a convention. The dominant convention is two's complement.

Two's-complement range

$$-2^{n-1} \text{ through } 2^{n-1} - 1$$

For n bits, positive values use ordinary binary. To encode $-x$: write x in n bits, invert all bits, and add 1.

Example: -5 using eight bits

$$5 = 00000101 \rightarrow \text{invert} = 11111010 \rightarrow \text{add } 1 = 11111011.$$

Why two's complement is useful

- The same binary addition circuit works for positive and negative values.
- There is one representation of zero.
- The MSB acts as a sign indicator, but its weight is -2^{n-1} .
- Overflow can be detected from sign inconsistencies.

Interpretation formula

$$\text{value} = -b_{n-1}2^{n-1} + \sum_{j=0}^{n-2} b_j 2^j.$$

Although quantum amplitudes are not stored as ordinary signed integers inside the mathematical state vector, two's complement remains important for classical controllers, fixed-point arithmetic, cryptographic implementations, and reversible arithmetic circuits.

6. Binary arithmetic

Binary addition uses the same place-value logic as decimal addition, but each column contains only 0 or 1.

a	b	carry-in	sum	carry-out
0	0	0	0	0
0	1	0	1	0
1	1	0	0	1
1	1	1	1	1

The one-bit equations are

$$\text{sum} = a \text{ XOR } b \text{ XOR } c_{\text{in}}$$

$$c_{\text{out}} = (a \text{ AND } b) \text{ OR } (c_{\text{in}} \text{ AND } (a \text{ XOR } b)).$$

Example

$$1011_2 + 0110_2 = 10001_2 \text{ (11 + 6 = 17).}$$

Subtraction

Subtraction can be performed using borrowing, but processors commonly add the two's complement of the subtrahend.

$$a - b = a + (-b).$$

Multiplication and shifts

Left shifting an unsigned bit string by k places multiplies by 2^k when no overflow occurs. Right shifting divides by powers of two, though signed right shifts require a convention.

$$001101 \ll 2 = 110100.$$

Binary arithmetic becomes especially important in reversible circuit design because intermediate carries and temporary values must be managed without destroying information.

7. Boolean algebra

Boolean algebra treats 0 and 1 as truth values and defines operations that produce new bits. Common notation includes NOT a , a AND b , a OR b , and a XOR b .

Key identities

Identity	Meaning
$a \text{ AND } 1 = a$	identity
$a \text{ OR } 0 = a$	identity
$a \text{ AND } 0 = 0$	annihilator
$a \text{ OR } 1 = 1$	annihilator
$a \text{ AND } a = a$	idempotence
$a \text{ OR } a = a$	idempotence
$a \text{ XOR } a = 0$	self-cancellation
$a \text{ XOR } 0 = a$	XOR identity
$\text{NOT}(\text{NOT } a) = a$	double negation

De Morgan's laws

$$\text{NOT}(a \text{ AND } b) = (\text{NOT } a) \text{ OR } (\text{NOT } b)$$

$$\text{NOT}(a \text{ OR } b) = (\text{NOT } a) \text{ AND } (\text{NOT } b).$$

Distributive laws

$$a \text{ AND } (b \text{ OR } c) = (a \text{ AND } b) \text{ OR } (a \text{ AND } c)$$

$$a \text{ OR } (b \text{ AND } c) = (a \text{ OR } b) \text{ AND } (a \text{ OR } c).$$

Boolean identities let us simplify logic circuits, reduce gate count, and prove equivalence between implementations.

8. Logic gates and truth tables

Core Boolean operations

NOT

a	out
0	1
1	0

AND

a	b	out
0	0	0
0	1	0
1	0	0
1	1	1

OR

a	b	out
0	0	0
0	1	1
1	0	1
1	1	1

XOR

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

NAND

a	b	out
0	0	1
0	1	1
1	0	1
1	1	0

XNOR

a	b	out
0	0	1
0	1	0
1	0	0
1	1	1

Figure 4. Truth tables completely specify Boolean gates.

A truth table lists every possible input and the corresponding output. An operation on k input bits has 2^k rows.

Universal gates

NAND alone and NOR alone are functionally complete: every Boolean function can be built using only one of these gate types. This is useful in physical circuit design because a technology can optimize a small universal primitive.

XOR and XNOR

XOR outputs 1 when the inputs differ. XNOR outputs 1 when they are equal. XOR is central to parity, error detection, stream ciphers, linear codes, and CNOT.

Logic levels and timing

An abstract truth table ignores voltage thresholds and propagation delays. Real circuits must account for finite switching time, noise margins, fan-out, and synchronization. These engineering details are distinct from the Boolean function itself.

9. XOR and arithmetic over GF(2)

XOR: inequality test and addition modulo 2

0 XOR 0	=	0	same
0 XOR 1	=	1	different
1 XOR 0	=	1	different
1 XOR 1	=	0	same

$$a \text{ XOR } b = (a + b) \bmod 2$$

Figure 5. XOR is addition modulo two.

The field GF(2) contains two elements, 0 and 1. Addition and multiplication are performed modulo 2:

Operation	Results
Addition	$0+0=0$, $0+1=1$, $1+1=0$
Multiplication	$0*0=0$, $0*1=0$, $1*1=1$

Therefore:

$$a + b \text{ in GF}(2) = a \text{ XOR } b$$

$$ab \text{ in GF}(2) = a \text{ AND } b.$$

Vectors over GF(2)

Bit strings can be treated as vectors in $\text{GF}(2)^n$. Vector addition is bitwise XOR. This viewpoint powers linear error-correcting codes, parity-check matrices, binary linear algebra, and many cryptographic constructions.

Example

$$10110 \text{ XOR } 01101 = 11011.$$

Unlike ordinary integer addition, XOR has no carries. That makes it linear over GF(2), but not over the real numbers.

10. Combinational circuits and adders

Binary addition is built from Boolean gates

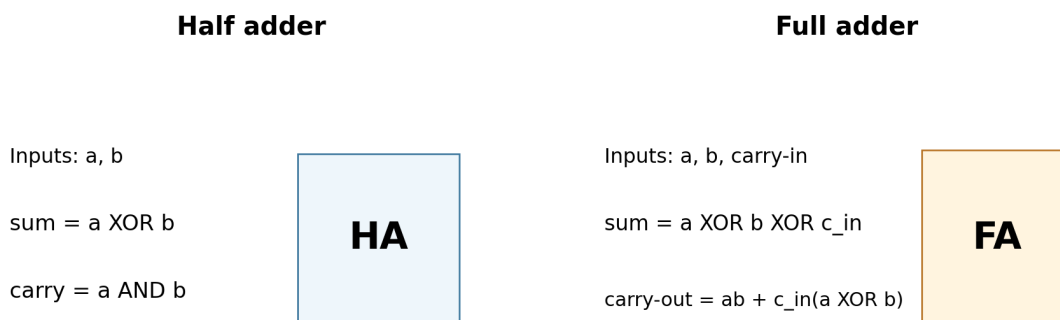


Figure 6. Half and full adders combine XOR, AND, and OR.

A combinational circuit produces outputs determined only by its current inputs. It has no memory. Adders, comparators, multiplexers, encoders, and decoders are common examples.

Half adder

$$\text{sum} = a \text{ XOR } b, \text{ carry} = a \text{ AND } b.$$

Full adder

$$\text{sum} = a \text{ XOR } b \text{ XOR } c_{\text{in}}$$

$$c_{\text{out}} = ab \text{ OR } c_{\text{in}}(a \text{ XOR } b).$$

Ripple-carry addition

An n-bit ripple-carry adder chains full adders so that each carry becomes the next stage's carry-in. The design is simple, but the carry may need to propagate through all n stages. Faster adders use carry-lookahead or prefix structures.

Quantum connection

A quantum arithmetic circuit must be reversible. It cannot simply overwrite inputs or discard carry information. Classical adder logic therefore has to be reorganized using reversible gates and often temporary ancilla qubits.

11. Bitwise operations, masks, and shifts

Bitwise operations apply a Boolean operation independently at every bit position.

$$101101 \text{ AND } 110011 = 100001$$

$$101101 \text{ OR } 110011 = 111111$$

$$101101 \text{ XOR } 110011 = 011110.$$

Masks

A mask selects, clears, sets, or toggles chosen positions.

Goal	Expression
Test selected bits	$x \text{ AND mask}$
Clear selected bits	$x \text{ AND (NOT mask)}$
Set selected bits	$x \text{ OR mask}$
Toggle selected bits	$x \text{ XOR mask}$

Shifts

- Logical left shift inserts zeros on the right.
- Logical right shift inserts zeros on the left.
- Arithmetic right shift usually replicates the sign bit.
- Rotations wrap displaced bits around rather than discarding them.

Cryptographic relevance

XOR, rotations, modular addition, substitution tables, and bit permutations are building blocks of symmetric cryptography. Careful implementations must also consider timing leakage and other side channels.

12. Hamming weight and Hamming distance

Three-bit strings form a Hamming cube

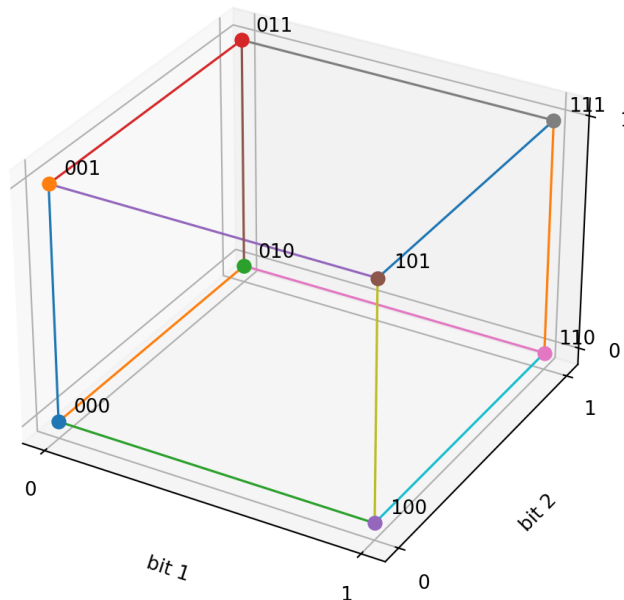


Figure 7. Adjacent vertices differ in exactly one bit.

The Hamming weight of a bit string is the number of ones:

$$\text{wt}(101101)=4.$$

The Hamming distance between equal-length strings is the number of positions where they differ:

$$d_H(x,y)=\text{wt}(x \text{ XOR } y).$$

Example

$$x=101101, y=111001, x \text{ XOR } y=010100, \text{ so } d_H(x,y)=2.$$

Why distance matters

- A code can detect up to $d_{\min}-1$ errors if its minimum codeword distance is $d_{\min}$.
- A code can correct $\text{floor}((d_{\min}-1)/2)$ errors.
- QKD estimates error rates by comparing subsets of classical measurement strings.
- Post-quantum code-based cryptography is built around hard problems involving noisy codewords and Hamming metrics.

13. Deterministic state transformations

A deterministic function maps each input state to exactly one output state:

$$f: \{0,1\}^n \rightarrow \{0,1\}^m.$$

The function is fully specified by a truth table with 2^n input rows. A classical circuit implements such a function.

State-transition viewpoint

For finite state spaces, a deterministic map can be represented by a matrix with one 1 in each input column. If the map is reversible, there is also exactly one 1 in each row, making the matrix a permutation matrix.

Sequential systems

A circuit with memory has an internal state s_t . Its update may be written

$$s_{t+1} = F(s_t, x_t), y_t = G(s_t, x_t).$$

Finite-state machines, counters, registers, and protocol controllers fit this model.

Deterministic does not mean reversible

AND is deterministic because every input has one output. It is irreversible because multiple inputs map to 0. Reversibility requires a one-to-one mapping, not merely predictability.

14. Reversible and irreversible computation

Reversibility is about whether the input can be reconstructed

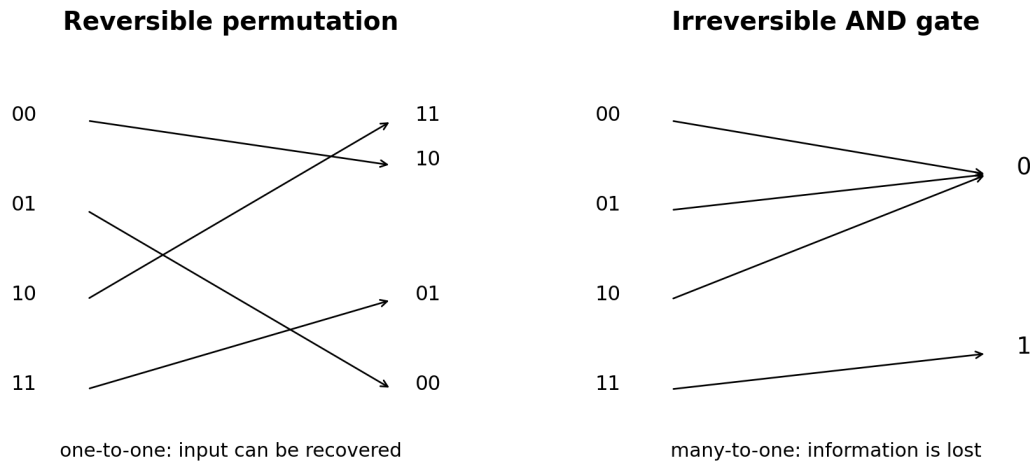


Figure 8. A reversible map is a permutation; an irreversible map merges inputs.

A function is reversible when the output uniquely determines the input. For finite sets, this means the function is bijective.

reversible: every input has one output and every output has one input.

Why ordinary gates are often irreversible

AND maps 00,01,10 to the same output 0. Once only the output is kept, the original input cannot be reconstructed. Erasing information also has a thermodynamic cost in idealized computation, summarized by Landauer's principle.

Embedding an irreversible function reversibly

Instead of replacing x by $f(x)$, preserve x and write the result into a target register:

$$(x,y) \rightarrow (x, y \text{ XOR } f(x)).$$

This transformation is reversible because applying it twice restores y . Quantum computation requires this style because closed-system quantum gates are unitary and therefore reversible.

15. Reversible gates: NOT, CNOT, Toffoli, and SWAP

Important reversible classical gates

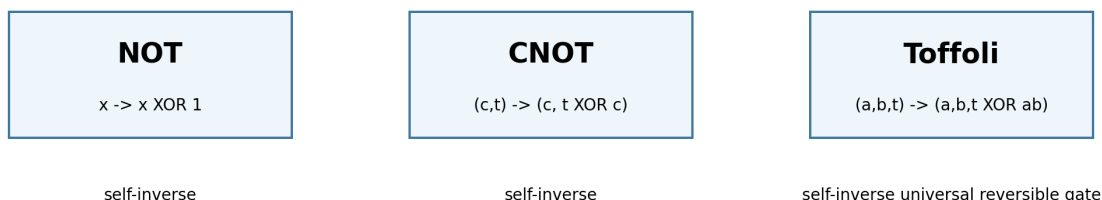


Figure 9. Reversible gates permute classical basis states.

NOT

$$x \rightarrow x \text{ XOR } 1.$$

CNOT

$$(c, t) \rightarrow (c, t \text{ XOR } c).$$

The control bit is unchanged. The target toggles only when $c=1$. On computational-basis inputs, quantum CNOT acts exactly like this reversible classical gate.

Toffoli or CCNOT

$$(a, b, t) \rightarrow (a, b, t \text{ XOR } (a \text{ AND } b)).$$

Toffoli can implement reversible classical computation and is universal when ancilla bits and NOT are available.

SWAP

$$(a, b) \rightarrow (b, a).$$

All these gates are self-inverse: applying the same gate twice restores the input. Their truth tables are permutations, so their matrix representations are permutation matrices and hence unitary.

16. Fan-out and copying

Classical information can be copied when the source is in a definite basis state. Starting with a target initialized to 0, CNOT gives

$$(x,0) \rightarrow (x,x).$$

This is classical fan-out. It does not create information from nothing because the original bit is retained and the second bit becomes correlated with it.

Why this is not universal quantum cloning

For an arbitrary qubit $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, applying CNOT to $|\psi\rangle|0\rangle$ produces

$$\alpha|00\rangle + \beta|11\rangle,$$

not

$$(\alpha|0\rangle + \beta|1\rangle) \otimes (\alpha|0\rangle + \beta|1\rangle).$$

The first result is generally entangled, not two independent copies. CNOT copies computational-basis values but cannot clone arbitrary unknown quantum states.

No-deleting and reversibility

Similarly, a reversible gate cannot erase an unknown input without moving its information elsewhere. These constraints are not obstacles to classical reversible computation, where temporary values can be uncomputed after use.

17. Random bits and probability distributions

A random bit is a Bernoulli random variable:

$$P(X=1)=p, P(X=0)=1-p.$$

A fair bit has $p=1/2$. A biased bit has p different from $1/2$.

Independent bit strings

If $X_1, \dots, X_n$ are independent fair bits, every n -bit string has probability

$$P(x)=2^{-n}.$$

Pseudorandom versus true random

- A pseudorandom generator deterministically expands a short seed into a longer sequence that should be computationally indistinguishable from random for a target application.
- A physical random generator samples unpredictable physical processes.
- Quantum random-number generators use quantum measurement, but require careful device modelling and statistical validation.

Min-entropy

For security, average uncertainty is not always sufficient. Min-entropy measures the probability of the most likely outcome:

$$H_{\infty}(X) = -\log_2(\max_x P(X=x)).$$

It controls how many nearly uniform secret bits can be extracted by privacy amplification.

18. Information and binary entropy

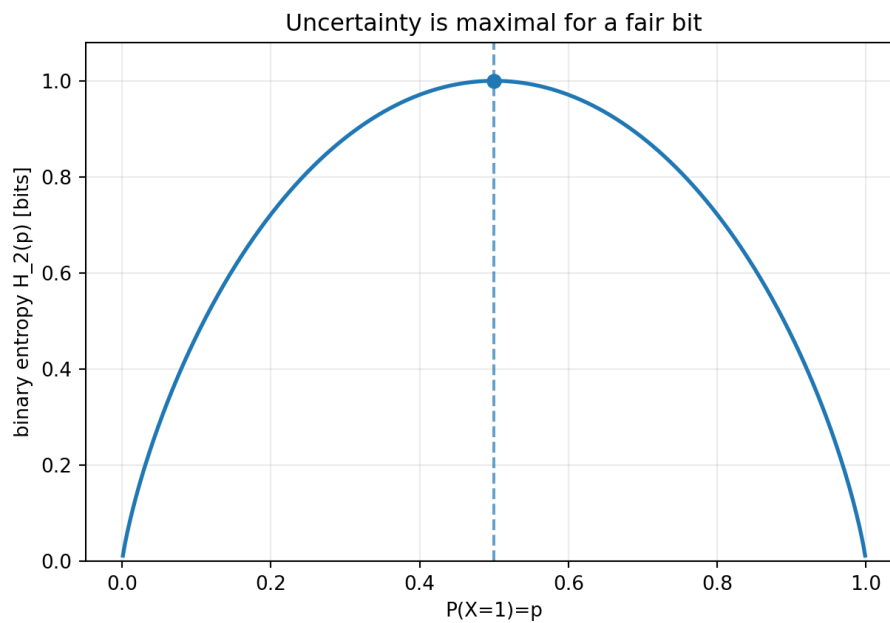


Figure 10. A deterministic bit has zero entropy; a fair bit has one bit of entropy.

Shannon entropy quantifies average uncertainty:

$$H(X) = -\sum_x P(x) \log_2 P(x).$$

For a Bernoulli bit with $P(1)=p$:

$$H_2(p) = -p \log_2 p - (1-p) \log_2 (1-p).$$

Interpretation

- $H_2(0)=H_2(1)=0$: the bit is predictable.
- $H_2(1/2)=1$: a fair bit carries one bit of uncertainty.
- Entropy is measured in bits because the logarithm base is 2.

Compression viewpoint

For long independent sequences, entropy gives the asymptotically optimal average number of binary digits needed per symbol in lossless compression. In QKD, entropy functions appear in key-rate bounds and error-correction leakage estimates.

19. Classical channels and noise

Binary symmetric channel: each bit flips with probability p

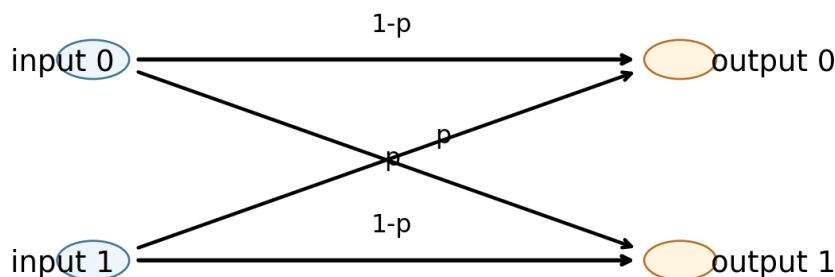


Figure 11. The binary symmetric channel flips either input with probability p .

A classical channel is described by conditional probabilities $P(Y=y|X=x)$. The binary symmetric channel (BSC) has

$$P(Y \neq X) = p, P(Y = X) = 1 - p.$$

If X is a fair input bit, the channel capacity is

$$C = 1 - H_2(p) \text{ bits per channel use.}$$

Other channel models

- Binary erasure channel: a bit is either delivered correctly or replaced by an erasure symbol.
- Asymmetric channel: $0 \rightarrow 1$ and $1 \rightarrow 0$ have different probabilities.
- Burst-error channel: errors occur in correlated clusters.

Quantum connection

Quantum hardware produces classical measurement strings through noisy readout channels. Readout mitigation often begins by estimating a classical confusion matrix. QKD also converts physical errors into classical bit-error statistics during parameter estimation.

20. Parity and repetition codes

Simple classical error detection and correction

Even parity bit	Three-bit repetition
data: 1 0 1 1	encode: 0 -> 000, 1 -> 111
number of ones = 3	received: 101
append parity 1 -> 1 0 1 1 1	majority vote -> 1

Figure 12. Parity detects errors; repetition adds redundancy for correction.

Parity

The parity of a string is the XOR of all bits:

$$\text{parity}(x) = x_1 \text{ XOR } x_2 \text{ XOR } \dots \text{ XOR } x_n.$$

An even-parity code appends one bit so the total number of ones is even. It detects any odd number of bit flips, but it cannot identify which bit changed and may miss an even number of errors.

Three-bit repetition code

$$0 \rightarrow 000, 1 \rightarrow 111.$$

One flipped bit can be corrected by majority vote. The codewords have Hamming distance 3, so the code corrects one error.

Syndromes

Linear codes describe valid strings using parity-check equations. The syndrome $s = Hx^T$ over $GF(2)$ is zero for valid codewords and identifies classes of errors. This classical idea generalizes directly to quantum stabilizer error correction, where syndrome bits reveal errors without directly measuring the logical quantum information.

21. Correlation and mutual information

Two bits may be statistically related even if each bit individually looks random. Correlation is about the joint distribution.

Examples

Relationship	Joint outcomes for fair X
$Y=X$	00 and 11, each with probability $1/2$
$Y=\text{NOT } X$	01 and 10, each with probability $1/2$
Independent	00,01,10,11, each with probability $1/4$

Mutual information

$$I(X;Y)=H(X)+H(Y)-H(X,Y)=H(X)-H(X|Y).$$

It measures how much learning Y reduces uncertainty about X.

- If X and Y are independent, $I(X;Y)=0$.
- If $Y=X$ and X is a fair bit, $I(X;Y)=1$ bit.
- Mutual information is symmetric: $I(X;Y)=I(Y;X)$.

Cryptographic meaning

Security often requires the adversary's information about a secret to be negligible. QKD analyses compare the legitimate parties' correlations with the information potentially available to an eavesdropper, then use privacy amplification to reduce that leakage.

22. Classical cryptographic primitives

Classical information operations appear in both conventional and post-quantum cryptography.

One-time pad

$$c = m \text{ XOR } k, m = c \text{ XOR } k.$$

With a uniformly random secret key as long as the message and used only once, the ciphertext provides perfect secrecy. XOR is its core operation.

Hash functions

A cryptographic hash maps an arbitrary-length bit string to a fixed-length digest. Desired properties include preimage resistance, second-preimage resistance, and collision resistance.

Message authentication

A message authentication code uses a shared secret key to verify integrity and authenticity. QKD still requires an authenticated classical channel; quantum mechanics alone does not authenticate the communicating parties.

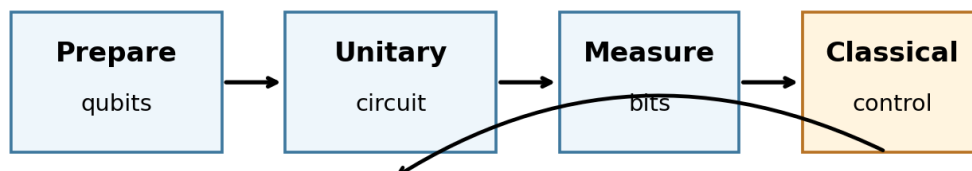
Post-quantum primitives

- Lattice-based schemes use polynomial and vector arithmetic modulo integers.
- Code-based schemes use noisy codewords and decoding hardness.
- Hash-based signatures rely primarily on secure hash functions.
- Multivariate and isogeny-based ideas use other algebraic structures, though practical status differs by scheme.

At implementation level, every key, ciphertext, nonce, seed, polynomial coefficient, and protocol message is represented by bits.

23. Classical control in quantum systems

Classical bits remain essential inside quantum algorithms



measurement results can condition later gates

Figure 13. Quantum processors are embedded in classical control loops.

A realistic quantum workflow contains several classical layers:

- Compile a high-level circuit into hardware-supported gates.
- Schedule pulses and calibrate control parameters.
- Measure qubits and digitize analog signals into classical outcomes.
- Decode errors or post-select data.
- Condition later operations on earlier measurement bits.
- Estimate probabilities, expectation values, and uncertainties from counts.

Dynamic circuits

A dynamic circuit may measure a qubit, store the outcome in a classical bit, and condition a later gate on that bit. Teleportation and quantum error correction rely on this quantum-classical interaction.

Example

measure $q \rightarrow$ classical bit c ; if $c=1$, apply X to another qubit.

The conditional branch is classical logic even though its effect controls quantum evolution.

24. Classical bits versus qubits

Classical and quantum information use different state descriptions

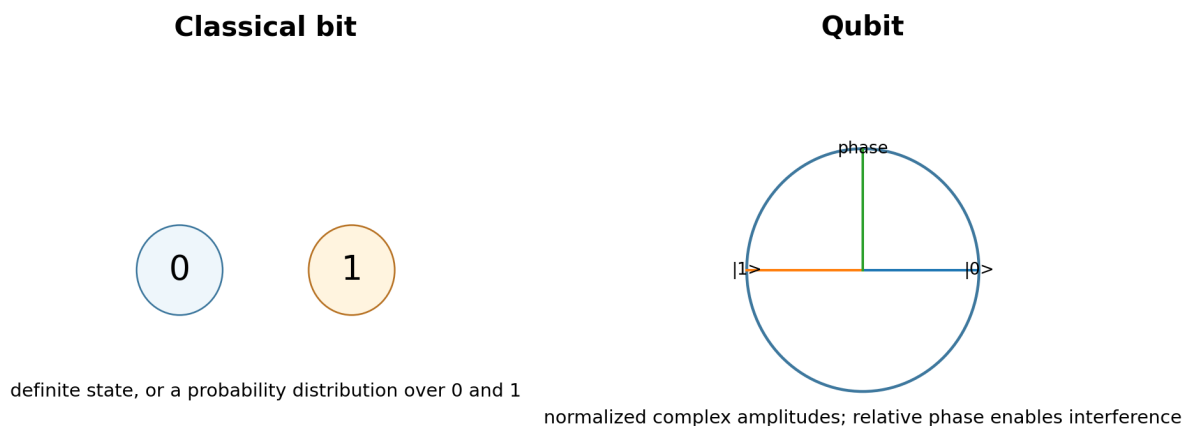


Figure 14. A qubit is not merely a continuously valued classical bit.

Classical bit	Qubit
Definite pure states 0 or 1	Pure state $\alpha 0\rangle + \beta 1\rangle$
Uncertainty described by probabilities	Amplitudes are complex and normalized
Copying a definite bit is allowed	Unknown arbitrary state cannot be cloned
Logic may be irreversible	Closed-system gates are unitary and reversible
No relative phase	Relative phase affects interference
Reading may be made effectively nondestructive	Measurement generally changes the state

Important nuance

A classical probabilistic bit and a qubit can produce the same probabilities in one measurement basis. They are nevertheless different because the qubit may retain coherence and relative phase, which can be revealed by measuring in another basis.

Computational basis bridge

On basis states, reversible quantum gates behave like classical permutations. The specifically quantum behaviour appears when those gates act linearly on superpositions and entangled states.

25. Worked examples

Example 1: binary conversion

Convert 173 to binary.

$$173=128+32+8+4+1, \text{ so } 173_{10}=10101101_2.$$

Example 2: XOR and Hamming distance

$$x=110101, y=100111; x \text{ XOR } y=010010; d_H(x,y)=2.$$

Example 3: reversible evaluation of AND

Use a target t and the Toffoli transformation:

$$(a,b,t) \rightarrow (a,b,t \text{ XOR } ab).$$

With $t=0$, the output target equals ab while a and b are preserved. Applying Toffoli again restores $t=0$.

Example 4: channel error rate

A binary symmetric channel has $p=0.08$. In 10,000 transmissions, the expected number of flips is 800 and the standard deviation is

$$\sqrt{Np(1-p)} = \sqrt{10000 \cdot 0.08 \cdot 0.92} \text{ approx } 27.1.$$

Example 5: entropy

For $P(1)=0.9$:

$$H_2(0.9) = -0.9 \log_2 0.9 - 0.1 \log_2 0.1 \text{ approx } 0.469 \text{ bits.}$$

26. Common mistakes

Mistake	Correction
Treating a bit string as unordered	Position determines value and meaning.
Confusing XOR with OR	XOR is 0 when both inputs are 1.
Assuming deterministic means reversible	A many-to-one deterministic map loses information.
Calling a probability mixture a superposition	A mixture has probabilities but no coherent phase.
Ignoring endianness	State-vector and printed-string ordering may differ.
Assuming CNOT clones arbitrary qubits	It copies basis values but entangles general superpositions.
Treating parity as full error correction	A single parity bit mostly detects; it does not locate errors.
Using Shannon entropy as a complete security guarantee	Operational security may require min-entropy and composable bounds.

A useful habit is to identify the mathematical category of every object: definite bit string, probability distribution, Boolean function, reversible permutation, or quantum state vector. Many confusions disappear once the category is explicit.

27. Exercises

- 1 Convert 219 to binary and hexadecimal.
- 2 Interpret 11101011 as an eight-bit two's-complement integer.
- 3 Compute $101101 \text{ XOR } 011110$, and find their Hamming distance.
- 4 Simplify $a \text{ XOR } a \text{ XOR } b \text{ XOR } 1$.
- 5 Construct the truth table for $(a \text{ AND } b) \text{ XOR } c$.
- 6 Add 10111_2 and 01101_2 .
- 7 Explain why AND is deterministic but not reversible.
- 8 Give the complete CNOT mapping for 00,01,10,11.
- 9 Show that Toffoli is self-inverse.
- 10 Encode 1 using the three-bit repetition code, then decode 101.
- 11 A BSC has $p=0.02$. What is the expected number of errors in 5000 bits?
- 12 Calculate $H_2(1/4)$ to three decimal places.
- 13 If X is a fair bit and $Y=X$, calculate $H(X)$, $H(X|Y)$, and $I(X;Y)$.
- 14 Describe how CNOT can copy a classical basis bit without violating no-cloning.
- 15 Explain why an authenticated classical channel is still needed in QKD.
- 16 For four bits, how many definite register states exist?
- 17 Design a mask that clears the lowest four bits of an eight-bit value.
- 18 Why is a permutation matrix a valid unitary transformation?
- 19 Compare a fair classical mixture of 0 and 1 with the qubit $|+\rangle$.
- 20 List three places classical information processing appears in a quantum experiment.

28. Selected solutions

- 1 $219=128+64+16+8+2+1$, so $11011011_2=16_{10}$.
- 2 Invert and add one: 11101011 represents -21 .
- 3 $101101 \text{ XOR } 011110 = 110011$, whose weight is 4; distance 4.
- 4 $a \text{ XOR } a=0$, so the expression is $b \text{ XOR } 1 = \text{NOT } b$.
- 5 Evaluate all eight triples; output is 1 exactly when c differs from ab .
- 6 $10111+01101=100100$ ($23+13=36$).
- 7 Each input has one output, but $00,01,10$ all map to 0, so the input cannot be recovered.
- 8 $00 \rightarrow 00, 01 \rightarrow 01, 10 \rightarrow 11, 11 \rightarrow 10$.
- 9 The control bits are unchanged and the target is XORed with ab twice; $x \text{ XOR } r \text{ XOR } r=x$.
- 10 $1 \rightarrow 111$; 101 has two ones, so majority decoding returns 1.
- 11 $Np=5000*0.02=100$ expected errors.
- 12 $H_2(1/4)$ approx 0.811 bits.
- 13 $H(X)=1$, $H(X|Y)=0$, and $I(X;Y)=1$ bit.
- 14 With target 0, CNOT maps $|x\rangle|0\rangle$ to $|x\rangle|x\rangle$ for x in $\{0,1\}$; arbitrary superpositions instead become entangled.
- 15 Without authentication, an attacker can impersonate the parties and run separate quantum sessions.
- 16 $2^4=16$ states.
- 17 Use mask 11110000 and compute $x \text{ AND } 11110000$.
- 18 Its columns are orthonormal basis vectors, so $P^T P=I$; over complex entries use $P^\dagger P=I$.
- 19 Both give 50/50 in the Z basis, but $|+\rangle$ is coherent and gives $+$ with certainty in the X basis.
- 20 Examples: compilation, pulse control, measurement digitization, feed-forward, decoding, and statistical estimation.

29. Reference sheet

Concept	Formula or fact
Number of n-bit strings	2^n
Unsigned value	$\sum_j b_j 2^j$
Two's-complement range	-2^{n-1} to $2^{n-1}-1$
XOR	$a \text{ XOR } b = (a+b) \bmod 2$
Hamming distance	$d_H(x,y) = \text{wt}(x \text{ XOR } y)$
Parity	XOR of all bits
Reversible map	bijection / permutation
CNOT	$(c,t) \rightarrow (c, t \text{ XOR } c)$
Toffoli	$(a,b,t) \rightarrow (a,b, t \text{ XOR } ab)$
Binary entropy	$H_2(p) = -p \log_2 p - (1-p) \log_2 (1-p)$
BSC capacity	$1 - H_2(p)$
Mutual information	$I(X;Y) = H(X) - H(X Y)$
One-time pad	$c = m \text{ XOR } k$
Classical basis states	$ x\rangle, x \in \{0,1\}^n$

Essential bridge to quantum computing

- Classical reversible gates are permutations of computational-basis strings.
- Quantum gates extend these maps linearly to superpositions.
- Measurement converts a quantum state into classical outcomes.
- Classical probability analyzes the resulting counts.
- Classical coding, authentication, estimation, and control remain indispensable in quantum protocols.

The next topic is Introduction to Qubits: state vectors, amplitudes, measurement, phase, and the Bloch-sphere parameterization.